

O mesmo componente, agora mais claro e acessível

Aula 5 · Interface móvel e acessibilidade

Programação para Dispositivos Móveis
IFMA Campus Itapecuru-Mirim
Prof. Dr. Thales Levi Azevedo Valente

A lógica já funciona. Hoje, a interface evolui.

Na Aula 4, garantimos que os dados e o comportamento estão corretos.
Hoje, focamos na experiência de quem usa.

- **O que permanece:** Props, Estado (useState), Eventos (onPress) e Coleções (.map).
- **O que evolui:** Estrutura visual, alinhamento, feedback de toque e semântica.

Recuperação ativa: encontre o que já conhece

Olhe para o código ao lado. Onde estão estes elementos da Aula 4?

- A definição das **Props**?
- O **onPress** que dispara a ação?
- O **setter** que altera o estado?

Desafio: Se mudarmos o nome da prop, o componente ainda funciona? Por quê?

```
type CartaoProps = {  
  memoria: Memoria;  
  selecionada: boolean;  
  aoSelecionar: () => void;  
};  
  
export function Cartao({  
  memoria, selecionada, aoSelecionar  
}: CartaoProps) {  
  return (  
    <Pressable onPress={aoSelecionar}>  
      <Text>{memoria.titulo}</Text>  
      <Text>{memoria.data}</Text>  
    </Pressable>  
  );  
}
```

Resposta — contrato, evento e estado estão conectados

As quatro respostas do slide anterior aparecem em pontos diferentes do código.

```
1 type CartaoProps = {  
  memoria: Memoria;  
  selecionada: boolean;  
  aoSelecionar: () => void;  
};  
  
export function Cartao({ memoria, selecionada, aoSelecionar }: CartaoProps) {  
  return (  
    <Pressable 2 onPress={aoSelecionar}>  
      <Text>{memoria.titulo}</Text>  
    </Pressable>  
  );  
}
```

```
<Cartao  
  memoria={memoria}  
  selecionada={idSelecionada === memoria.id}  
  4 aoSelecionar={() =>  
    3 setIdSelecionada(memoria.id)  
  }  
>
```

1 • **Props** — definem o contrato de entrada do componente

2 • **onPress** — liga o toque à função recebida

3 • **setter** — altera o estado no componente pai

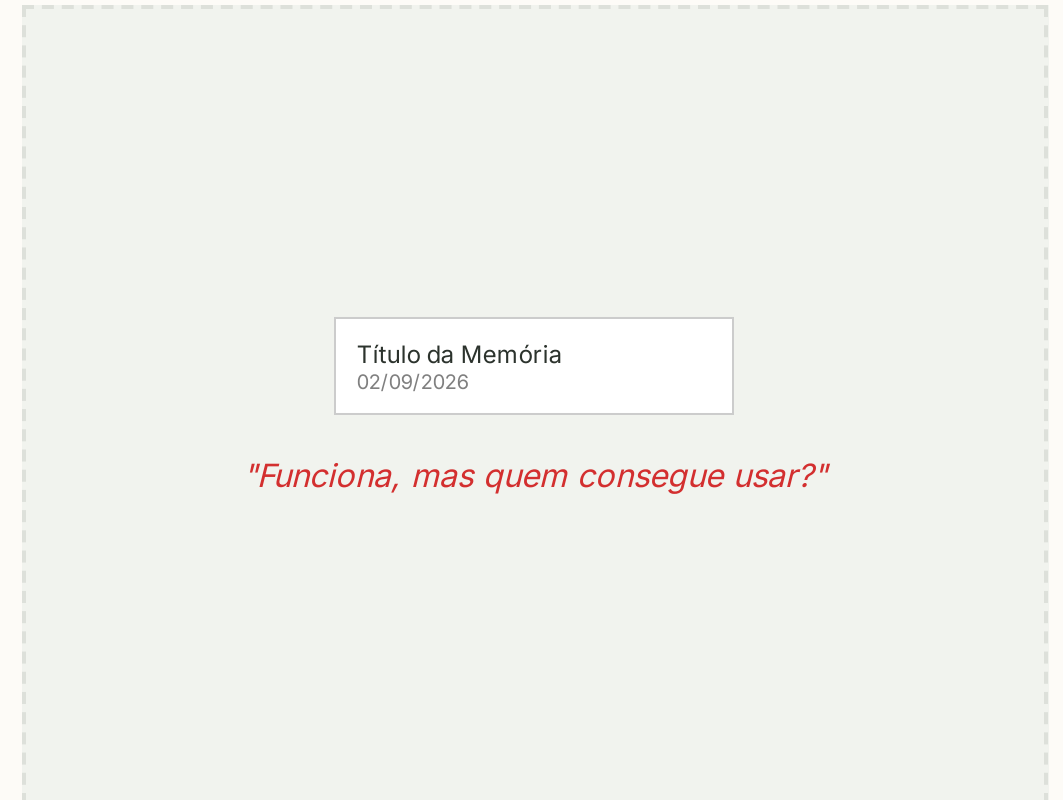
4 • **a prop aoSelecionar** leva a mudança de estado até o cartão

CartaoProps → desestruturação no Cartao → uso no JSX /
onPress → <Cartao ... /> no componente pai →
setIdSelecionada(...)

Desafio: renomear uma prop em apenas um ponto quebra o contrato. Renomear de forma consistente no tipo, no componente e em quem o utiliza mantém o comportamento.

Mesma função; três barreiras observáveis

- 1 Difícil de Ler**
Textos amontoados, sem hierarquia visual ou respiro.
- 2 Difícil de Tocar**
Área de interação pequena e sem resposta visual ao toque.
- 3 Difícil de Compreender**
O que é título? O que é ação? O leitor de tela entende?



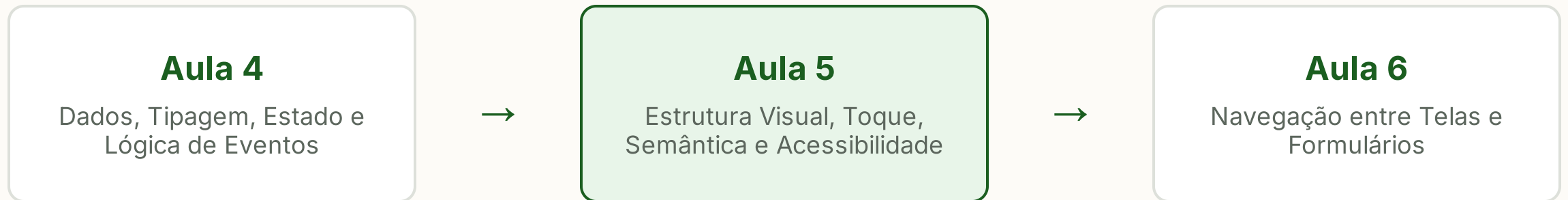
A tela principal organiza a coleção; o cartão organiza cada item

- A tela mantém a coleção e o estado da seleção.
- **map()** continua transformando cada dado em interface.
- Agora cada item é mostrado pelo componente **CartaoMemoria**.

**Quem continua dono do
idSelecionada: a tela ou o cartão?**

```
<View style={styles.lista}>
  {memorias.map((memoria) => (
    <CartaoMemoria
      key={memoria.id}
      memoria={memoria}
      selecionada={memoria.id === idSelecionada}
      aoSelecionar={() => setIdSelecionada(memoria.id)}
    />
  ))}
</View>
```

Aula 4 cuidou do comportamento. Aula 5 cuida da experiência.



Antes do estilo, precisamos enxergar a tela como uma árvore

TelaPrincipal (View)

- Título da tela (Text)
- Lista de memórias (View)
 - CartaoMemoria
 - Imagem
 - Título
 - Data
 - Ação

container = elemento que agrupa e organiza outros elementos

filhos = elementos que ficam dentro do container

Quem organiza o cartão: a tela ou o próprio cartão?

Passo 1 — dar estrutura de leitura

Agrupamos elementos relacionados para que o sistema (e o usuário) entenda a estrutura.

- **View:** Agrupa o conteúdo do cartão.
- **Text:** Diferencia título de descrição.
- **Lógica:** Note que as props `memoria` continuam as mesmas.

```
return (  
  <Pressable onPress={aoSelecionar}>  
    <View>  
      <Text style={styles.titulo}>  
        {memoria.titulo}  
      </Text>  
      <Text style={styles.data}>  
        {memoria.data}  
      </Text>  
    </View>  
  </Pressable>  
);
```

Passo 3 — nomear decisões visuais

- **StyleSheet.create()** tira detalhes visuais do meio do JSX.
- Nomes como titulo, data e imagem tornam o código mais legível.
- A IDE consegue ajudar melhor com propriedades e valores válidos.
- O mesmo estilo pode ser reutilizado.

```
const styles = StyleSheet.create({  
  titulo: { fontSize: 20, fontWeight: '700' },  
  data: { fontSize: 14, color: '#5C665D' },  
  imagem: { width: 72, height: 72 },  
});
```

Microcheck 1 — Previsão

O que muda com padding e gap?

Se adicionarmos padding: 20 ao container e gap: 10 entre os filhos, o que o usuário verá?

A)
Elementos
maiores

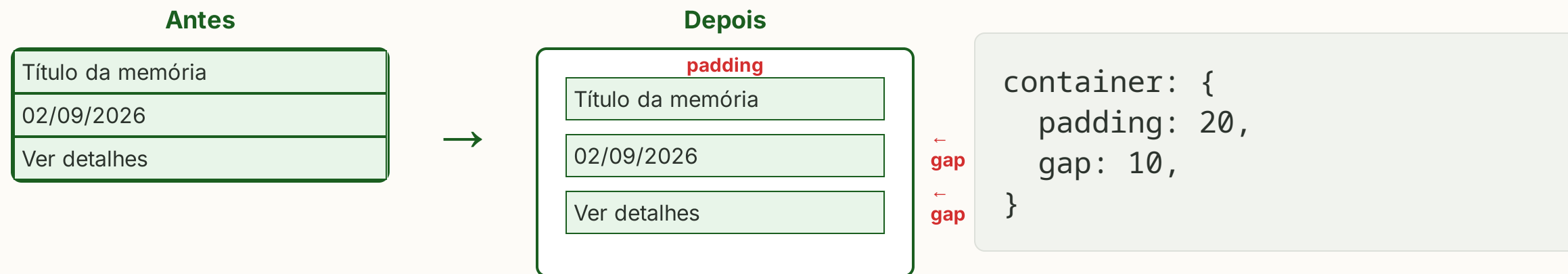
B) Mais espaço
interno e entre itens

C) Mudança
nas cores

(Pense por 30 segundos antes de responder)

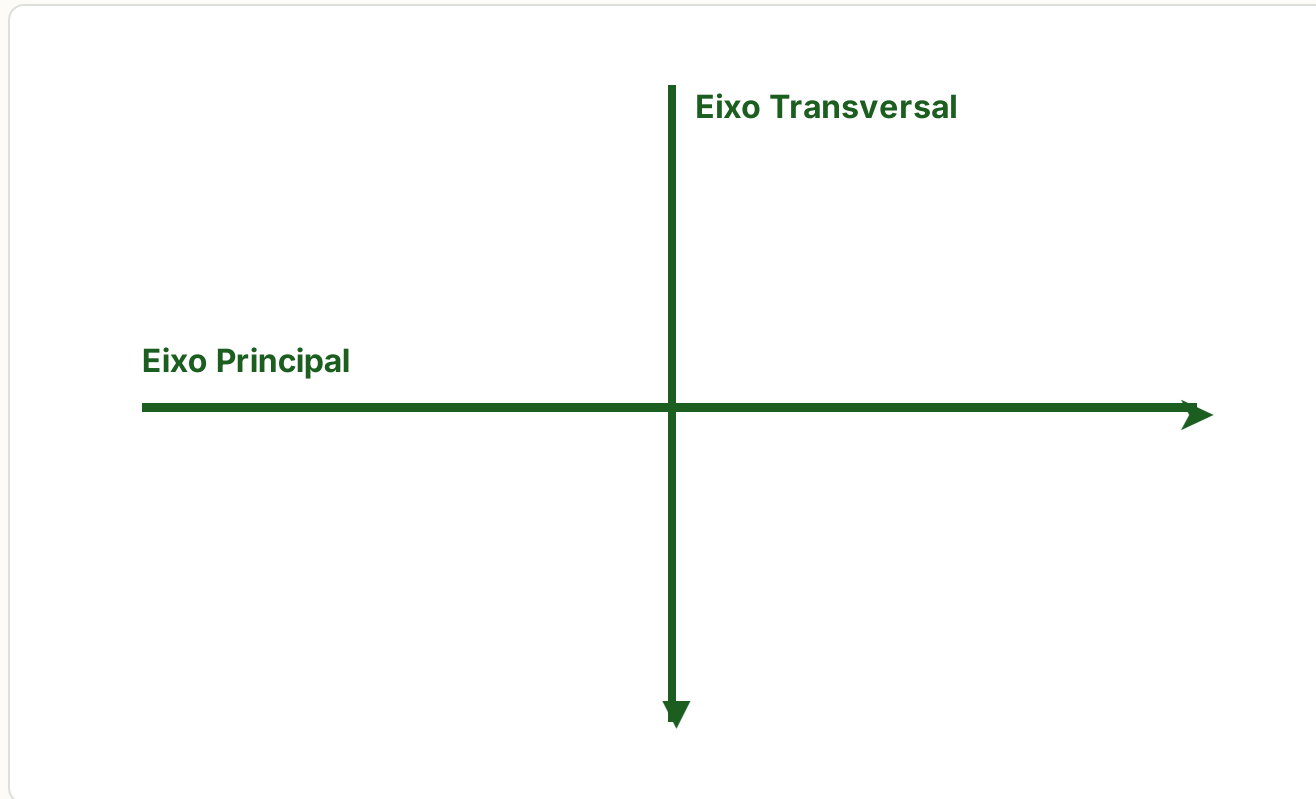
Resposta — padding afasta da borda; gap separa os filhos

B · Mais espaço interno e entre itens



padding cria espaço entre a borda do container e seus filhos. gap cria espaço entre os filhos.

Flexbox responde primeiro a uma pergunta: em qual direção os filhos se organizam?



eixo principal = direção escolhida para distribuir os filhos

eixo transversal = direção perpendicular ao eixo principal

justifyContent atua no eixo principal

alignItems atua no eixo transversal

Previsão: row ou column?

flexDirection: 'column' (Padrão)



Itens empilhados verticalmente.

flexDirection: 'row'



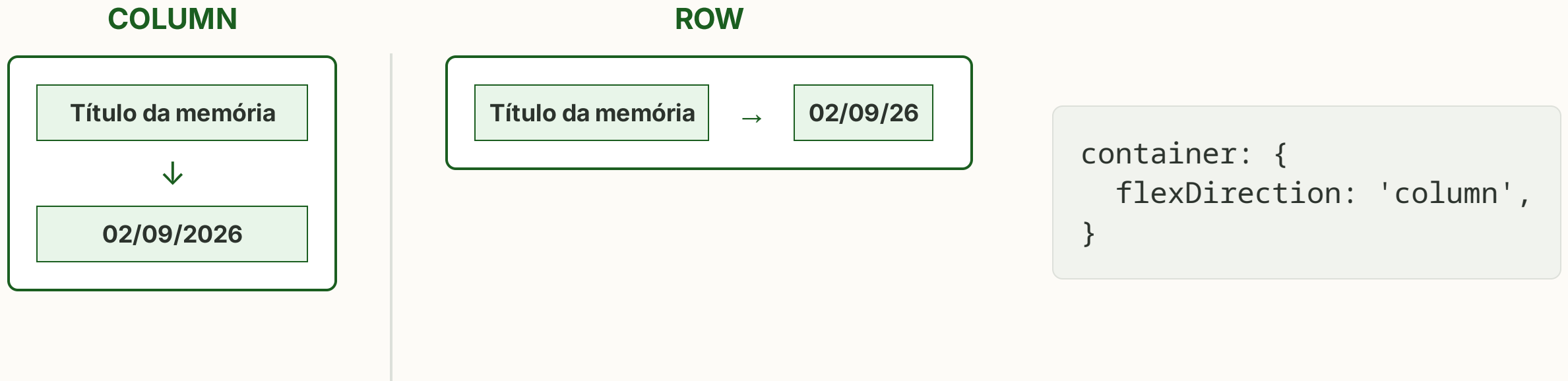
Itens alinhados horizontalmente.

Desafio: Se o nosso cartão tem o título em cima e a data embaixo, qual direção estamos usando?

Dica: No React Native, o padrão é sempre column!

Resposta — título em cima e data embaixo formam uma column

`flexDirection: 'column'`



Em column, o eixo principal é vertical. Os filhos são organizados de cima para baixo.

Microcheck 2 — Onde cada propriedade atua?

Relacione a propriedade ao seu eixo

`justifyContent`

Eixo Principal

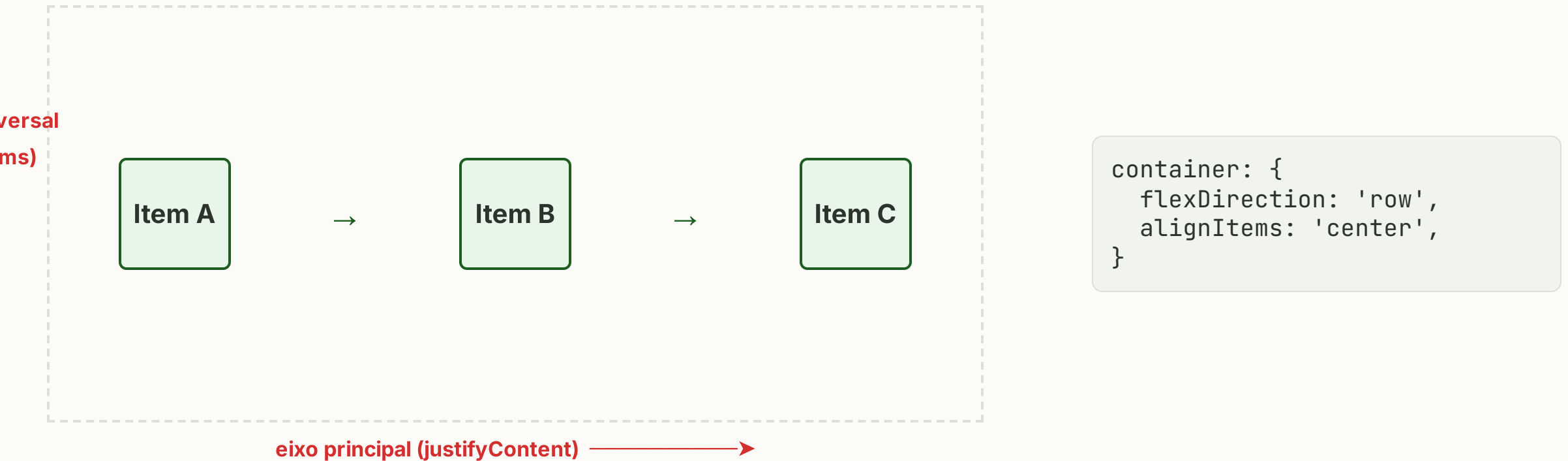
`alignItems`

Eixo Transversal

Pergunta: Se `flexDirection` for row, qual propriedade centraliza verticalmente?

Resposta — com row, alignItems atua verticalmente

alignItems: 'center'



Com `flexDirection: 'row'`, o eixo principal é horizontal. Por isso, `alignItems` atua no eixo vertical.

`justifyContent` acompanha o eixo principal. `alignItems` acompanha o eixo transversal.

Passo 2 — imagem só entra quando comunica alguma coisa

- Nesta prática, usaremos apenas um asset local fictício fornecido no projeto-base.
- Definimos width e height para controlar o layout do cartão.
- Imagem informativa recebe **accessibilityLabel** útil.
- Imagem decorativa não deve repetir uma informação que já está em texto.

Nota: as dimensões são uma decisão de layout da prática.

```
<Image
  source={require('/home/ubuntu/projects/aulas-ifma-
6b697bf4/9f968abb-a5aa-4c42-a20a-175c31ded2c1.png')}
  style={styles.imagem}
  accessibilityLabel="Ilustração da memória"
/>
```

onPress existe. Mas onde eu toco?

O Pressable da Aula 4 já funciona, mas agora vamos torná-lo **perceptível**.

- **Affordance:** A aparência deve convidar ao toque.
- **Feedback:** O usuário precisa saber que o toque foi registrado.

```
// A lógica da Aula 4 permanece:  
<Pressable  
  onPress={aoSelecionar}  
  style={styles.container}  
>  
  {/* Conteúdo do cartão */}  
</Pressable>
```

Prever → executar → observar o pressed

O Pressable nos dá o estado pressed em tempo real.

- **Estilo Dinâmico:** Mudamos a aparência apenas enquanto o dedo está na tela.
- **Opacidade:** Um feedback clássico e eficaz.

```
<Pressable
  onPress={aoSelecionar}
  style={({ pressed }) => [
    styles.container,
    { opacity: pressed ? 0.5 : 1 }
  ]}
>
  { /* ... */ }
</Pressable>
```

Área tocável não é detalhe: dedos precisam de espaço

- Referência Android para elementos acionáveis: pelo menos **48 × 48 dp**.
- Evite ações coladas umas nas outras.
- **hitSlop** pode ampliar a área detectável, mas não substitui um controle visualmente claro.

Android Developers

```
acao: {  
    minWidth: 48,  
    minHeight: 48,  
    paddingHorizontal: 16,  
}
```

Acessibilidade: quatro perguntas de qualidade

1. Consigo Perceber?

A informação está visível? O contraste é suficiente?
Depende só de cor?

2. Consigo Operar?

O botão é grande o suficiente? O toque funciona? Consigo navegar?

3. Consigo Compreender?

O texto é claro? A ação é óbvia? O que acontece se eu clicar?

4. A tecnologia entende?

O leitor de tela sabe que isso é um botão? O rótulo faz sentido?

Semântica: diga o papel do elemento e dê um nome útil à ação

- **accessibilityRole** informa o tipo de controle.
- **accessibilityLabel** pode dar um nome acessível mais contextual.
- **accessibilityHint** só é necessário quando o resultado da ação não fica claro pelo rótulo.

```
<Pressable
  onPress={aoSelecionar}
  accessibilityRole="button"
  accessibilityLabel={`Ver detalhes de ${memoria.titulo}`}
>
```

Microcheck 3 — qual rótulo identifica melhor a ação?

Contexto: A tela possui vários cartões. Em todos eles, o texto visual da ação é apenas "Detalhes".

Qual **accessibilityLabel** ajuda melhor uma pessoa que usa leitor de tela a saber QUAL memória será aberta?

A accessibilityLabel="Botão"

B accessibilityLabel="Detalhes"

C accessibilityLabel="Ver detalhes da memória Casa de Farinha"

Escolha a alternativa e explique o que falta nas outras duas.

Resposta — o rótulo precisa identificar a ação no contexto

C · `accessibilityLabel="Ver detalhes da memória Casa de Farinha"`

O QUE APARECE VISUALMENTE

Memória: Casa de Farinha

Detalhes

Memória: Festa do Divino

Detalhes

```
<Pressable
  accessibilityRole="button"
  accessibilityLabel={
    `Ver detalhes da memória ${memoria.titulo}`
  }
>
```

LEITURA POUCO INFORMATIVA

"Detalhes, botão" ... "Detalhes, botão"

LEITURA CONTEXTUAL

"Ver detalhes da memória Casa de Farinha, botão"

`accessibilityRole` informa o papel do controle. `accessibilityLabel` dá um nome útil e contextual à ação.

Cor ajuda; não pode carregar o estado sozinha

Problema: Só Cor



O que este círculo vermelho significa? Erro? Atenção? Selecionado?

Solução: Cor + Texto + Ícone



Atenção: Selecionado

Agora a informação é clara mesmo para quem não distingue cores.

Regra de Ouro: Nunca use apenas a cor para comunicar algo importante.

- Use rótulos de texto.
- Use ícones ou formas.
- Garanta contraste (mínimo 4.5:1).

A mesma lógica; uma interface transformada

Aula 4

Título da Memória
02/09/2026



Aula 5



Título da Memória
02/09/2026

Ver detalhes

O que mudou: Hierarquia, Espaçamento, Feedback e Semântica.

O que NÃO mudou: As props, o estado e a função de clique.

Demonstração: cinco passes, um componente

- 1 Estrutura (View/Text)
- 2 Estilos (StyleSheet)
- 3 Layout (Flexbox)
- 4 Interação (Pressed)
- 5 Acessibilidade (Role/Label)

Acompanhe no telão!

Vamos transformar o código da Aula 4 passo a passo, prevendo cada mudança.

Oficina em dupla: checkpoints A → D

A. Estrutura (10 min)

- View/Text organizados
- Estilos no StyleSheet
- Lógica preservada

B. Layout e Imagem (12 min)

- Flexbox aplicado
- Espaçamento coerente
- Asset local validado

C. Toque e Acessibilidade (15 min)

- Feedback pressed
- Alvo de toque 48dp
- Semântica básica

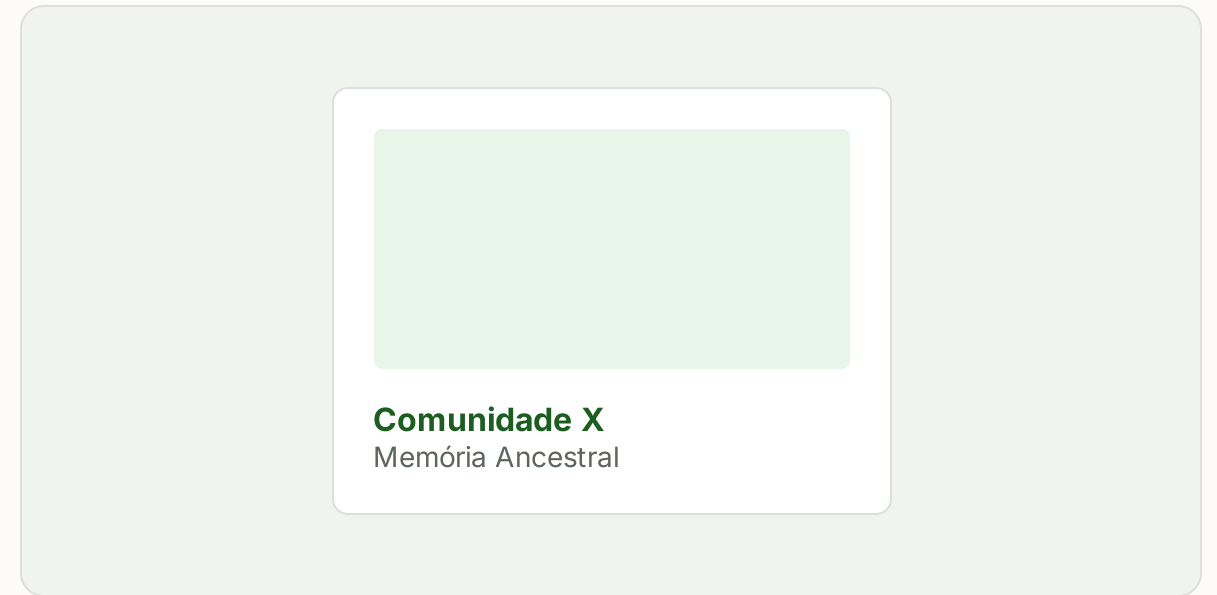
D. Reuso e Explicação (10 min)

- Componente consolidado
- Props tipadas
- Defesa da decisão

Transferência A — Memórias Quilombolas

Evolua o cartão de memórias da Aula 4:

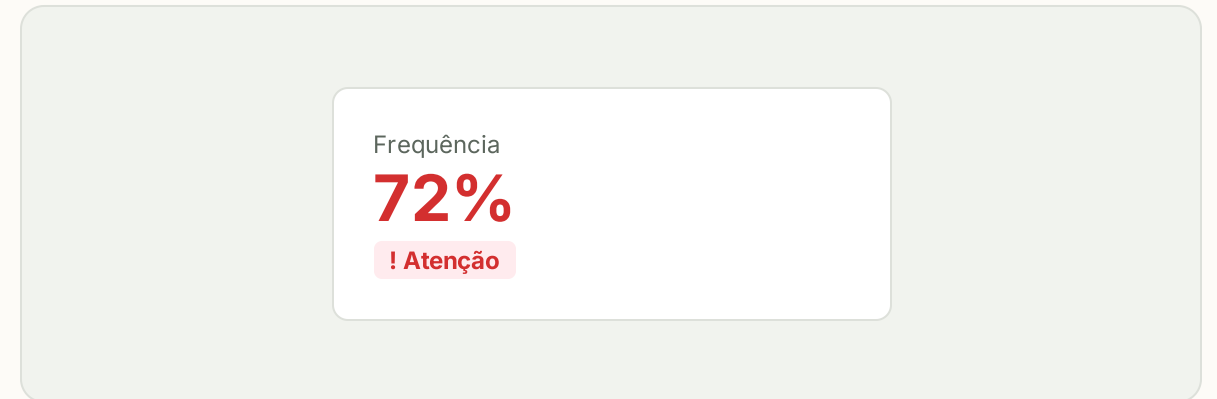
- **Título e Comunidade:** Hierarquia clara.
- **Imagem:** Use uma ilustração ou foto local.
- **Ação:** "Ver detalhes" com feedback.
- **Acessibilidade:** Rótulo útil e contextual.



Transferência B — EDA / permanência

Evolua o indicador de permanência da Aula 4:

- **Indicador e Valor:** Destaque numérico.
- **Situação:** Não dependa apenas de cor (ex: Atenção).
- **Layout:** Flexbox para alinhar valor e descrição.
- **Acessibilidade:** Role e label para o estado.



Teste por pares: 60 segundos sem editar

Heurística Rápida

- ☐ O que parece tocável?
- ☐ O toque produz resposta perceptível?
- ☐ Consigo ler tudo sem esforço?
- ☐ Algum estado depende apenas de cor?
- ☐ O alvo de toque parece adequado?

Ação Obrigatória

A dupla autora deve registrar **UMA barreira** identificada pelo colega e realizar **UMA correção** imediata no código.

Evidência: commit + explicação + ticket

Ticket de Saída

- Realize o **commit** final da aula.
- Aponte uma **decisão visual** que você tomou.
- Aponte uma **decisão de acessibilidade**.
- Explique quem se beneficia dessas escolhas.

Agora o componente está pronto para se conectar

Aula 6 · Navegação e formulários

- Expo Router
- lista → detalhe → retorno
- formulário validado
- nova memória ou check-in/registro de ação

Hoje organizamos como a pessoa vê e toca. Na próxima aula, vamos organizar para onde ela vai e o que ela pode informar.

Referências oficiais — React Native

React Native · Core Components and Native Components · reactnative.dev/docs/intro-react-native-components

React Native · View · reactnative.dev/docs/view

React Native · Text · reactnative.dev/docs/text

React Native · Image · reactnative.dev/docs/image

React Native · Pressable · reactnative.dev/docs/pressable

React Native · StyleSheet · reactnative.dev/docs/stylesheet

React Native · Layout with Flexbox · reactnative.dev/docs/flexbox

React Native · Accessibility · reactnative.dev/docs/accessibility

Referências oficiais — Android e WCAG

| **Android Developers** · Make apps more accessible · developer.android.com/guide/topics/ui/accessibility/apps

| **Android Developers** · Minimum touch target size · developer.android.com/guide/topics/ui/accessibility/apps#large-controls

| **W3C WCAG 2.2** · Success Criterion 1.4.1 Use of Color · w3.org/WAI/WCAG22/Understanding/use-of-color.html

| **W3C WCAG 2.2** · Success Criterion 1.4.3 Contrast (Minimum) · w3.org/WAI/WCAG22/Understanding/contrast-minimum.html

| **Material Design 3** · Accessibility · m3.material.io/foundations/accessibility